

Iterazione

Spesso abbiamo bisogno che una determinata operazione/istruzione venga eseguita più volte all'interno del nostro codice. Sebbene alcune volte sia fattibile riscrivere la stessa istruzione più volte (sommare ad una variabile un determinato valore per 2 volte), molte volte non lo è (sommare ad una variabile un determinato valore per 1000 volte).

In generale ogni qualvolta abbiamo bisogno di ripetere l'esecuzione di un blocco di istruzioni dobbiamo usare il costrutto dell'iterazione, ovvero un ciclo.

Il linguaggio C mette a disposizione 3 tipi di cicli:

- `while` → iterazione con controllo in testa
- `do... while` → iterazione con controllo in coda
- `for` → tipicamente usato quando è noto a priori il numero di volte che il ciclo verrà ripetuto.

while

Ciclo iterativo con controllo in testa, ovvero prima del blocco di istruzioni.

Sintassi:

```
while(condizione){  
    // istruzioni  
}
```

Le istruzioni verranno eseguite fintanto che la condizione risulta vera (si "esce" quando la condizione è falsa).

Dal momento che il controllo è effettuato in testa, se la condizione in partenza è falsa il blocco di istruzioni non viene eseguito neanche una volta.

do... while

Ciclo iterativo con controllo in coda, ovvero dopo il blocco di istruzioni.

Sintassi:

```
do{  
    // istruzioni  
} while(condizione);
```

Anche in questo caso le istruzioni verranno eseguite fintanto che la condizione risulta vera (si "esce" quando la condizione è falsa).

Dal momento che il controllo è effettuato in coda, le istruzioni verranno eseguite comunque almeno una volta, anche se la condizione è falsa in partenza.

for

Ciclo iterativo tipicamente usato quando è noto a priori il numero di volte che il ciclo verrà ripetuto.

Sintassi:

```
for(espressione1; espressione2; espressione3){  
    //istruzioni  
}
```

Funzionamento:

- l'espressione1 viene eseguita una sola volta prima dell'esecuzione del blocco di istruzioni → inizializzazione della variabile usata per ciclare
- l'espressione2 definisce la condizione di ripetizione del ciclo (se è vera il ciclo viene ripetuto)
- l'espressione3 viene eseguita ogni volta dopo l'esecuzione del blocco di istruzioni

Tutti e tre i tipi di cicli possono essere annidati (si possono avere cicli dentro altri cicli).

break

L'istruzione 'break' l'abbiamo già incontrata parlando dello switch, e abbiamo visto come questa ci consentisse di uscire da tale blocco dopo aver eseguito le istruzioni presenti in uno dei casi dello switch stesso.

Questa istruzione può essere utilizzata anche per uscire da un ciclo.

Nell'esempio di seguito, quando la variabile i vale 5 il ciclo viene interrotto.

```
for(int i = 0; i < 10; i++){  
    if (i == 5)  
        break;  
    printf("%d\n", i);  
}
```

continue

L'istruzione 'continue' serve per interrompere l'esecuzione del ciclo ad una determinata iterazione o quando una determinata condizione è verificata, saltando direttamente all'iterazione successiva.

Esempio:

```
for(int i = 0; i < 10; i++){  
    if (i == 5)  
        continue;  
    printf("%d\n", i);  
}
```

```
printf("%d\n", n);  
}
```

In questo caso, quando la variabile *i* vale 5 il ciclo salterà all'iterazione successiva senza proseguire con le istruzioni presenti nel ciclo.